微算機系統實習

第12組專案報告

LAB 08

組別： 12
班級、姓名與學號：
資工三 112590450 李馥亘
資工三 112590452 林希芸
資工三 112590454 林妍蓁
資工三 112590455 吳哲丞
日期：2025.12.09

1. 實驗內容：

- 練習撰寫可掛載於 Linux 系統中的字元裝置驅動(Character Device Driver)
- 以 file_operations 實作 open、release、read、write、ioctl 等核心函式
- 使用 Linux Kernel 檔案操作方式(vfs_read、vfs_write)控制 TX2 GPIO 腳位
- 透過 mknod 建立 /dev/demo 裝置節點
- 使用 C++ 程式(test.cpp)呼叫驅動模組以控制 4 顆 LED 或讀取 LED 狀態

2. 實驗過程及結果

實驗過程

- **驅動程式設計（demo.c）**
  - 建立 file_operations 結構，並依規定在 open、read、write、release、ioctl 中輸出：
    - Enter Open function
    - Enter Release function
    - Enter Read function
    - Enter Write function
    - Enter I/O Control function
  - 使用 Linux Kernel 檔案讀寫 API 操作
    - /sys/class/gpio/export
    - /sys/class/gpio/gpioX/direction
    - /sys/class/gpio/gpioX/value
  - 將四顆 LED 對應 GPIO：396、466、397、398，載入模組時一併 export 及 direction=out
- **Write 指令解析與 LED 控制**
  - 在 write 函式中以 sscanf("LED%d %s"）解析指令
  - Kernel 端控制對應 GPIO value = 1 或 0
- **Read LED 狀態**
  - 如果輸入只有 LEDx，驅動會讀取 /value 檔案並回傳：
    - 例如：LED1 Status: 1

- **使用者層 LED 控制程式（test.cpp）**
  - 透過 open("/dev/demo", O_WRONLY）寫入驅動
  - 指令格式：
    - ./Lab8 LED1 on
    - ./Lab8 LED3 off
    - ./Lab8 LED2（讀取狀態）

預期實驗結果

- 模組能成功載入、卸載並顯示初始化訊息
- Write 指令能控制四顆 LED 個別開關
- Read 指令能正確印出 LED 狀態
- 開啟 /dev/demo 時能印出 5 個規定訊息
- 驅動程序運作順暢且不造成 kernel error

實際上的結果

make demo.c

```
nvidia@nvidia-desktop:~/lab08$ ls
demo.c  main  Makefile  test.cpp
nvidia@nvidia-desktop:~/lab08$ make
make -C /lib/modules/4.9.201-tegra/build M=/home/nvidia/lab08 modules
make[1]: Entering directory '/usr/src/linux-headers-4.9.201-tegra-ubuntu18.04_aarch64/kernel-4.9'
  CC [M]  /home/nvidia/lab08/demo.o
/home/nvidia/lab08/demo.c: In function ‘drv_write’:
/home/nvidia/lab08/demo.c:138:5: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
     int ledNum = 0;
     ^~~
/home/nvidia/lab08/demo.c:150:5: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
     int matched = sscanf(user_cmd, "LED%d %9s", &ledNum, action);
     ^~~
/home/nvidia/lab08/demo.c:157:5: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
     int gpio = leds[ledNum - 1];
     ^~~
/home/nvidia/lab08/demo.c: In function ‘drv_read’:
/home/nvidia/lab08/demo.c:185:5: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
     int len = strlen(read_buf);
     ^~~
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/nvidia/lab08/demo.mod.o
  LD [M]  /home/nvidia/lab08/demo.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.9.201-tegra-ubuntu18.04_aarch64/kernel-4.9'
nvidia@nvidia-desktop:~/lab08$
```

sudo inmod demo.ko

```
[ 4714.320644] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.932465] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.932550] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951025] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951213] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951377] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951444] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951540] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.951630] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.952188] configfs-gadget gadget: Wrong NDP SIGN
[ 4720.952267] configfs-gadget gadget: Wrong NDP SIGN
[ 4721.309864] configfs-gadget gadget: Wrong NDP SIGN
[ 4721.371769] configfs-gadget gadget: Wrong NDP SIGN
[ 4721.371938] configfs-gadget gadget: Wrong NDP SIGN
[ 4727.272449] demo: started
[ 4727.982356] configfs-gadget gadget: Wrong NDP SIGN
[ 4727.982496] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000196] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000331] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000529] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000595] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000691] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.000865] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.001190] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.001265] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.313399] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.313755] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.421856] configfs-gadget gadget: Wrong NDP SIGN
[ 4728.422023] configfs-gadget gadget: Wrong NDP SIGN
nvidia@nvidia-desktop:~/lab08$
```

LED 基本開關控制實作

字元裝置驅動控制 LED 與狀態讀取

遇到的問題&問題如何解決

- **無法在 dmesg 輸出看見對應 Kernel log**
  - 在對驅動模組執行對應操作後，使用 dmesg 指令無法看見應輸出的系統訊息
  - 檢查後發現是使用了錯誤的輸出函式，導致內容無法顯示於 kernel log 中
  - 解決方法：將輸出函式改為 printk，系統訊息便正常顯示於 dmesg 輸出中

3. 程式碼：

demo.c

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/string.h>
#include <linux/slab.h>

#define MAJOR_NUM 60
#define MODULE_NAME "demo"

static char user_cmd[100];
static char read_buf[100];

/* LED 對應的 GPIO 腳位 */
static int leds[4] = {396, 466, 397, 398};

/* ===== 用檔案方式操作 GPIO ===== */
static int gpio_export_file(int gpio)
{
   struct file *fp;
   mm_segment_t old_fs;
   loff_t pos = 0;
   char buf[8];

   snprintf(buf, sizeof(buf), "%d", gpio);

   old_fs = get_fs();
   set_fs(KERNEL_DS);

   fp = filp_open("/sys/class/gpio/export", O_WRONLY, 0);
   if (IS_ERR(fp)) {
```

```
        set_fs(old_fs);
        printk("Failed to open /sys/class/gpio/export\n");
        return -1;
    }

    vfs_write(fp, buf, strlen(buf), &pos);
    filp_close(fp, NULL);
    set_fs(old_fs);
    return 0;
}

static int gpio_set_direction_file(int gpio, int output)
{
    struct file *fp;
    mm_segment_t old_fs;
    loff_t pos = 0;
    char path[64], buf[4];

    snprintf(path, sizeof(path),
"/sys/class/gpio/gpio%d/direction", gpio);
    snprintf(buf, sizeof(buf), "%s", output ? "out" : "in");

    old_fs = get_fs();
    set_fs(KERNEL_DS);

    fp = filp_open(path, O_WRONLY, 0);
    if (IS_ERR(fp)) {
        set_fs(old_fs);
        printk("Failed to open direction file\n");
        return -1;
    }

    vfs_write(fp, buf, strlen(buf), &pos);
    filp_close(fp, NULL);
    set_fs(old_fs);
```

```
    return 0;
}

static int gpio_set_value_file(int gpio, int value)
{
    struct file *fp;
    mm_segment_t old_fs;
    loff_t pos = 0;
    char path[64], buf[2];

    snprintf(path, sizeof(path),
"/sys/class/gpio/gpio%d/value", gpio);
    snprintf(buf, sizeof(buf), "%d", value);

    old_fs = get_fs();
    set_fs(KERNEL_DS);

    fp = filp_open(path, O_WRONLY, 0);
    if (IS_ERR(fp)) {
        set_fs(old_fs);
        printk("Failed to open value file\n");
        return -1;
    }

    vfs_write(fp, buf, strlen(buf), &pos);
    filp_close(fp, NULL);
    set_fs(old_fs);
    return 0;
}

static int gpio_get_value_file(int gpio)
{
    struct file *fp;
    mm_segment_t old_fs;
    loff_t pos = 0;
```

```
    char path[64], buf[2];
    int val = -1;

    snprintf(path, sizeof(path),
"/sys/class/gpio/gpio%d/value", gpio);

    old_fs = get_fs();
    set_fs(KERNEL_DS);

    fp = filp_open(path, O_RDONLY, 0);
    if (IS_ERR(fp)) {
        set_fs(old_fs);
        printk("Failed to open value file for read\n");
        return -1;
    }

    vfs_read(fp, buf, 1, &pos);
    filp_close(fp, NULL);
    set_fs(old_fs);

    if (buf[0] == '0') val = 0;
    else if (buf[0] == '1') val = 1;

    return val;
}

/* ===== LED 控制 ===== */
static void led_control(int led_index, int onoff)
{
    int gpio = leds[led_index];
    gpio_set_value_file(gpio, onoff);
}

/* ===== 驅動 write ===== */
static ssize_t drv_write(struct file *flip, const char *buf,
```

```
size_t count, loff_t *ppos)
{
    printk("Enter Write function\n");

    int ledNum = 0;
    char action[10] = {0};

    if (count >= sizeof(user_cmd))
        count = sizeof(user_cmd) - 1;

    if (copy_from_user(user_cmd, buf, count))
        return -EFAULT;

    user_cmd[count] = '\0'; // 字串結尾

    /* 解析 LED 指令 */
    int matched = sscanf(user_cmd, "LED%d %9s", &ledNum,
action);

    if (ledNum < 1 || ledNum > 4) {
        printk("Invalid LED number\n");
        return count;
    }

    int gpio = leds[ledNum - 1];

    if (matched == 2) {
        if (strcmp(action, "on") == 0) {
            led_control(ledNum - 1, 1);
        }
        else if (strcmp(action, "off") == 0) {
            led_control(ledNum - 1, 0);
        } else {
            printk("Unknown action: %s\n", action);
        }
```

```
    }
    else {
        /* 讀取 LED 狀態 */
        int val = gpio_get_value_file(gpio);
        snprintf(read_buf, sizeof(read_buf), "LED%d Status:
%d\n", ledNum, val);
    printk("LED%d Status: %d\n", ledNum, val);
    }

    return count;
}

/* ===== 驅動 read ===== */
static ssize_t drv_read(struct file *filp, char *buf, size_t
count, loff_t *ppos)
{
    printk("Enter Read function\n");

    int len = strlen(read_buf);

    if (*ppos > 0)
        return 0;

    if (copy_to_user(buf, read_buf, len))
        return -EFAULT;

    *ppos = len;
    return len;
}

static int drv_open(struct inode *inode, struct file *filp)
{
    printk("Enter Open function\n");
    return 0;
}
```

```
static int drv_release(struct inode *inode, struct file
*filp)
{
   printk("Enter Release function\n");
   return 0;
}
static long drv_ioctl(struct file *filp, unsigned int cmd,
unsigned long arg)
{
   printk("Enter I/O Control function\n");
   return 0;
}

static struct file_operations drv_fops = {
   .read = drv_read,
   .write = drv_write,
   .unlocked_ioctl = drv_ioctl,
   .open = drv_open,
   .release = drv_release,
};

/* ===== 模組初始化 ===== */
static int demo_init(void)
{
   int i;

   if (register_chrdev(MAJOR_NUM, MODULE_NAME, &drv_fops) <
0) {
       printk("%s: can't get major %d\n", MODULE_NAME,
MAJOR_NUM);
       return -EBUSY;
   }

   /* 初始化 GPIO: export + 設輸出 + 預設關閉 */
   for (i = 0; i < 4; i++) {
```

```
        gpio_export_file(leds[i]);
        gpio_set_direction_file(leds[i], 1); // output
        gpio_set_value_file(leds[i], 0);     // 預設關閉
    }

    printk("%s: started\n", MODULE_NAME);
    return 0;
}

/* ===== 模組移除 ===== */
static void demo_exit(void)
{
    unregister_chrdev(MAJOR_NUM, MODULE_NAME);
    printk("%s: removed\n", MODULE_NAME);
}

module_init(demo_init);
module_exit(demo_exit);

MODULE_LICENSE("GPL");
```

test.cpp

```
#include <iostream>
#include <string>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

using namespace std;

void setGPIO(string led, string status)
{
    int io = open("/dev/demo", O_WRONLY);
```

```
    if (io < 0) {
        perror("open error");
        return;
    }

    char buf[64] = {0};

    // 組合字串成：LED1 on
    if (status == "") {
        // 讀取 LED 狀態
        string cmd = led;
        strcpy(buf, cmd.c_str());
    } else {
        // 控制 LED on/off
        string cmd = led + " " + status;
        strcpy(buf, cmd.c_str());
    }

    cout << "Send to driver: " << buf << endl;

    write(io, buf, strlen(buf));    // 寫入 /dev/demo
    close(io);                      // 關閉
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        cout << "Usage: " << argv[0] << " LEDx [on/off]" <<
endl;
        return 0;
    }

    string led = argv[1];
    string status = "";
```

```
    if (argc == 3)
        status = argv[2];    // on/off

    setGPIO(led, status);

    return 0;
}
```

4. 本次實驗過程說明與解決方法：

本次實驗的主要過程是建立一個字元裝置驅動程式，並在驅動中配置 file_operations 的五大函式(open、read、write、release、ioctl)，再透過 Linux Kernel 的檔案操作指令（vfs_read()、vfs_write()等）來控制 GPIO 狀態。實作中以 demo.c 完成 LED 的 on/off 控制，以及在 read 函式中回傳 LED 目前的狀態，並使用 test.cpp 撰寫使用者層程式測試驅動功能是否正常。過程中也確認模組掛載、裝置節點建立及讀寫行為是否符合預期，最終成功完成驅動與應用層的整合操作。

5. 分工：

學號、組員	貢獻比例	工作內容
112590450 李馥亘	25%	小組報告、程式規劃
112590452 林希芸	25%	資料查詢、文書處理
112590454 林妍蓁	25%	實驗環境準備、實驗設計
112590455 吳哲丞	25%	測試與除錯、實驗實作